



PHENIKAA UNIVERSITY
Faculty of Computer Science

Software Analysis and Design

Module 5: Structural Modeling

Objectives



- ❖ Understand class diagrams and object diagrams.
- ❖ Be able to create class diagrams.
- ❖ Understand the relationship among classes.
- ❖ Understand the relationship between the structural and functional models.

- ❖ Structural models
- ❖ Object identification
- ❖ Class diagrams
- ❖ Structural models using class diagram

- ❖ All object-oriented systems development approaches are use-case driven, architecture-centric, and iterative and incremental
- ❖ To fully capture and represent the requirements for a business information system, we will have to iterate across all three architectural views (functional, structural, and behavioral)
- ❖ From an architecture-centric perspective, structural modeling supports the creation of an internal structural or static view of a business information system in that it shows how the system is structured to support the underlying business processes

- ❖ A structural model is a formal way of representing the objects that are used and created by a business system. It illustrates people, places, or things about which information is captured and how they are related to one another.
- ❖ The structural model is drawn using an iterative process in which the model becomes more detailed and less conceptual over time.
 - ❖ In analysis, analysts draw a conceptual model, which shows the logical organization of the objects without indicating how the objects are stored, created, or manipulated
 - ❖ In design, analysts evolve the conceptual structural model into a design model that reflects how the objects will be organized in databases and software

Structural Models



- ❖ Every time a systems analyst encounters a new problem to solve, the analyst must learn the underlying problem domain. The goal of the analyst is to discover the key objects contained in the problem domain and to build a structural model.
- ❖ Structural models represent the things, ideas, or concepts contained in the domain of the problem. They also allow the representation of the relationships among the things, ideas, or concepts.
- ❖ By creating a structural model of the problem domain, the analyst creates the vocabulary necessary for the analyst and users to communicate effectively.
- ❖ Structural models are depicted using class diagrams, and, in some cases, object diagrams

Element of Structural Models: Classes



- ❖ A class is a general template that we use to create specific instances, or objects, in the problem domain. All objects of a given class are identical in structure and behavior but contain different data in their attributes
- ❖ Two general kinds of classes of interest during analysis: concrete and abstract
 - ❖ To describe the application domain classes, we create concrete classes
 - ❖ Abstract classes do not actually exist in the real world; they are simply useful abstractions
- ❖ **Some kinds of classes, but at this point, we are interested only in domain classes**

Element of Structural Models: Attributes



- ❖ An attribute of an analysis class represents a piece of information that is relevant to the description of the class within the application domain. An attribute contains information the analyst or user feels the system should keep track of.
 - ❖ For example, a possible relevant attribute of an employee class is employee name, whereas one that might not be as relevant is hair color..
- ❖ Only attributes that are primitive or atomic types (i.e., integers, strings, doubles, date, time, Boolean, etc.) should be added.
- ❖ Most complex or compound attributes are really placeholders for relationships between classes. They should be modeled as relationships, not as attributes

Element of Structural Models: Operations



- ❖ The behavior of an analysis class is defined in an operation or service. In implementation phases, the operations are converted to methods.
- ❖ At this point, we use the term operation to describe the actions to which the instances of the class are capable of responding. Only problem domain–specific operations that are relevant to the problem should be considered.
 - ❖ For example, it is normally required that classes provide means of creating instances, deleting instances, accessing individual attribute values, setting individual attribute values, accessing individual relationship values, and removing individual relationship values. However, at this point, the analyst should avoid cluttering up the definition of the class with these basic types of operations and focus only on relevant problem domain–specific operations

Element of Structural Models: Relationships



- ❖ Relationships can be classified into three basic categories of data abstraction mechanisms: generalization relationships, aggregation relationships, and association relationships.
- ❖ These data-abstraction mechanisms allow the analyst to focus on the important dimensions while ignoring nonessential dimensions. As with attributes, the analyst must be careful to include only relevant relationships.

Element of Structural Models: Relationships



- ❖ Generalization Relationships: The generalization abstraction enables the analyst to create classes that inherit attributes and operations of other classes.
- ❖ Aggregation Relationships: all aggregation relationships relate parts to wholes or assemblies. For our purposes, we use the a-part-of or has-parts semantic relationship to represent the aggregation abstraction.
 - ❖ For example, a door is a-part-of a car, an employee is a-part-of a department, or a department is a-part-of an organization.

Element of Structural Models: Relationships



- ❖ Association Relationships: these relationships are usually a weaker form of the aggregation relationship.
- ❖ For example, a patient schedules an appointment. It could be argued that a patient is a-part-of an appointment. However, there is a clear semantic difference between this type of relationship and one that models the relationship between doors and cars or even workers and unions. Thus, they are simply considered to be associations between instances of classes.

- ❖ Structural models
- ❖ Object identification
- ❖ Class diagrams
- ❖ Structural models using class diagram

- ❖ Different approaches have been suggested to aid the analyst in identifying a set of candidate objects for the structural model. The three most common approaches are textual analysis, brainstorming, and patterns.
- ❖ Most analysts use a combination of these techniques to make sure that no important objects and object attributes, operations, and relationships have been overlooked

- ❖ The analyst performs textual analysis by reviewing the use-case diagrams and examining the text in the use-case descriptions to identify potential objects, attributes, operations, and relationships.
 - ❖ The nouns in the use case suggest possible classes,
 - ❖ The verbs suggest possible operations.

Textual Analysis Guideline



- ❖ A common or improper noun implies a class of objects (Danh từ chung thể hiện một lớp)
- ❖ A proper noun or direct reference implies an instance of a class (Danh từ riêng thể hiện một đối tượng)
- ❖ A collective noun implies a class of objects made up of groups of instances of another class.
- ❖ An adjective implies an attribute of an object (Tính từ thể hiện một thuộc tính)
- ❖ A doing verb implies an operation (Động từ thể hiện một thao tác)
- ❖ A being verb implies a classification relationship between an object and its class
- ❖ A having verb implies an aggregation or association relationship.
- ❖ A transitive verb implies an operation.
- ❖ An intransitive verb implies an exception.
- ❖ A predicate or descriptive verb phrase implies an operation.
- ❖ An adverb implies an attribute of a relationship or an operation.

Textual Analysis: Example



Use Case Name: Make Old Patient Appt		ID: 2	Importance Level: Low
Primary Actor: Old Patient		Use Case Type: Detail, Essential	
Stakeholders and Interests: Old Patient – wants to make, change, or cancel an appointment Doctor – wants to ensure patient's needs are met in a timely manner			
Brief Description: This use case describes how we make an appointment as well as changing or canceling an appointment for a previously seen patient.			
Trigger: Patient calls and asks for a new appointment or asks to cancel or change an existing appointment.			
Type: External			
Relationships: Association: Old Patient Include: Extend: Update Patient Information Generalization: Manage Appointments			
Normal Flow of Events: 1. The Patient contacts the office regarding an appointment. 2. The Patient provides the Receptionist with his or her name and address. 3. If the Patient's information has changed Execute the Update Patient Information use case. 4. If the Patient's payment arrangements has changed Execute the Make Payments Arrangements use case. 5. The Receptionist asks Patient if he or she would like to make a new appointment, cancel an existing appointment, or change an existing appointment. If the patient wants to make a new appointment, the S-1: new appointment subflow is performed. If the patient wants to cancel an existing appointment, the S-2: cancel appointment subflow is performed. If the patient wants to change an existing appointment, the S-3: change appointment subflow is performed. 6. The Receptionist provides the results of the transaction to the Patient.			
SubFlows: S-1: New Appointment 1. The Receptionist asks the Patient for possible appointment times. 2. The Receptionist matches the Patient's desired appointment times with available dates and times and schedules the new appointment. S-2: Cancel Appointment 1. The Receptionist asks the Patient for the old appointment time. 2. The Receptionist finds the current appointment in the appointment file and cancels it. S-3: Change Appointment 1. The Receptionist performs the S-2: cancel appointment subflow. 2. The Receptionist performs the S-1: new appointment subflow.			
Alternate/Exceptional Flows: S-1, 2a1: The Receptionist proposes some alternative appointment times based on what is available in the appointment schedule. S-1, 2a2: The Patient chooses one of the proposed times or decides not to make an appointment.			

- ❖ Identify potential objects: old patient, doctor, appointment, patient, office, receptionist, name, address, patient information, payment, date, and time.
- ❖ Identify potential operations that can be associated with the identified objects.
 - ❖ patient contacts office, makes a new appointment, cancels an existing appointment, changes an existing appointment, matches requested appointment times and dates with requested times and dates, and finds current appointment.

- ❖ Brainstorming is a process that a set of individuals sitting around a table suggest potential classes that could be useful for the problem. A brainstorming session is kicked off by a facilitator who asks the set of individuals
 - ❖ For example, using the appointment problem, the facilitator could ask the development team and users to think about their experiences of making appointments and to identify candidate classes. Potential objects come to mind: doctors, nurses, receptionists, appointment, illness, treatment, prescriptions, insurance card, and medical records.
- ❖ The participants should discuss and select which of the candidate objects should be considered further. Further brainstorming can take place to identify potential attributes, operations, and relationships for each of the identified objects.

- ❖ A pattern is simply a useful group of collaborating classes that provide a solution to a commonly occurring problem. Because patterns provide a solution to commonly occurring problems, they are reusable.
- ❖ By reusing these existing patterns of classes, we can more quickly and more completely define the system than if we start with a blank piece of paper.
- ❖ Many types of patterns have been proposed, ranging from high-level business-oriented patterns to more low-level design patterns

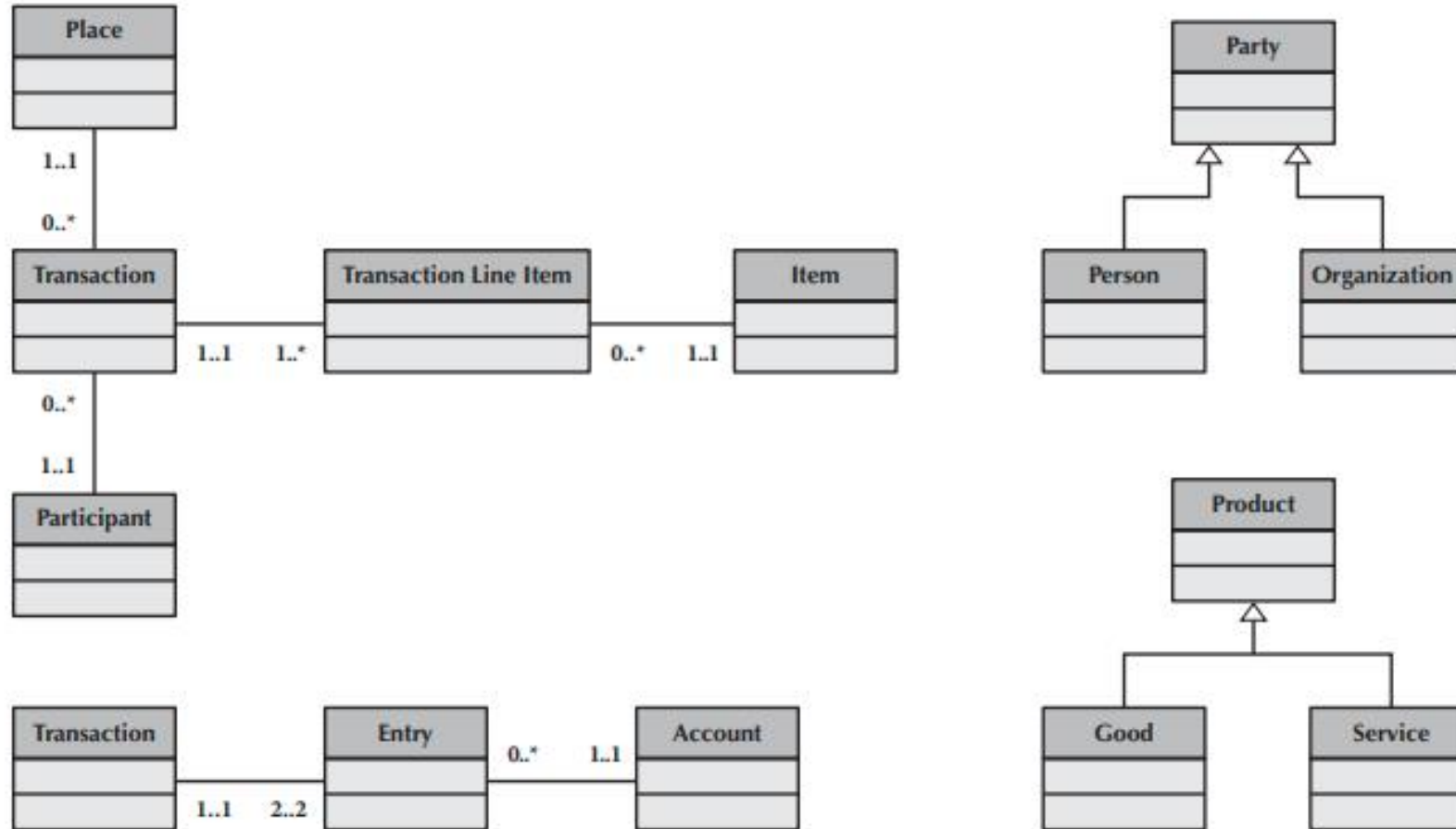


FIGURE 5-3 Sample Patterns

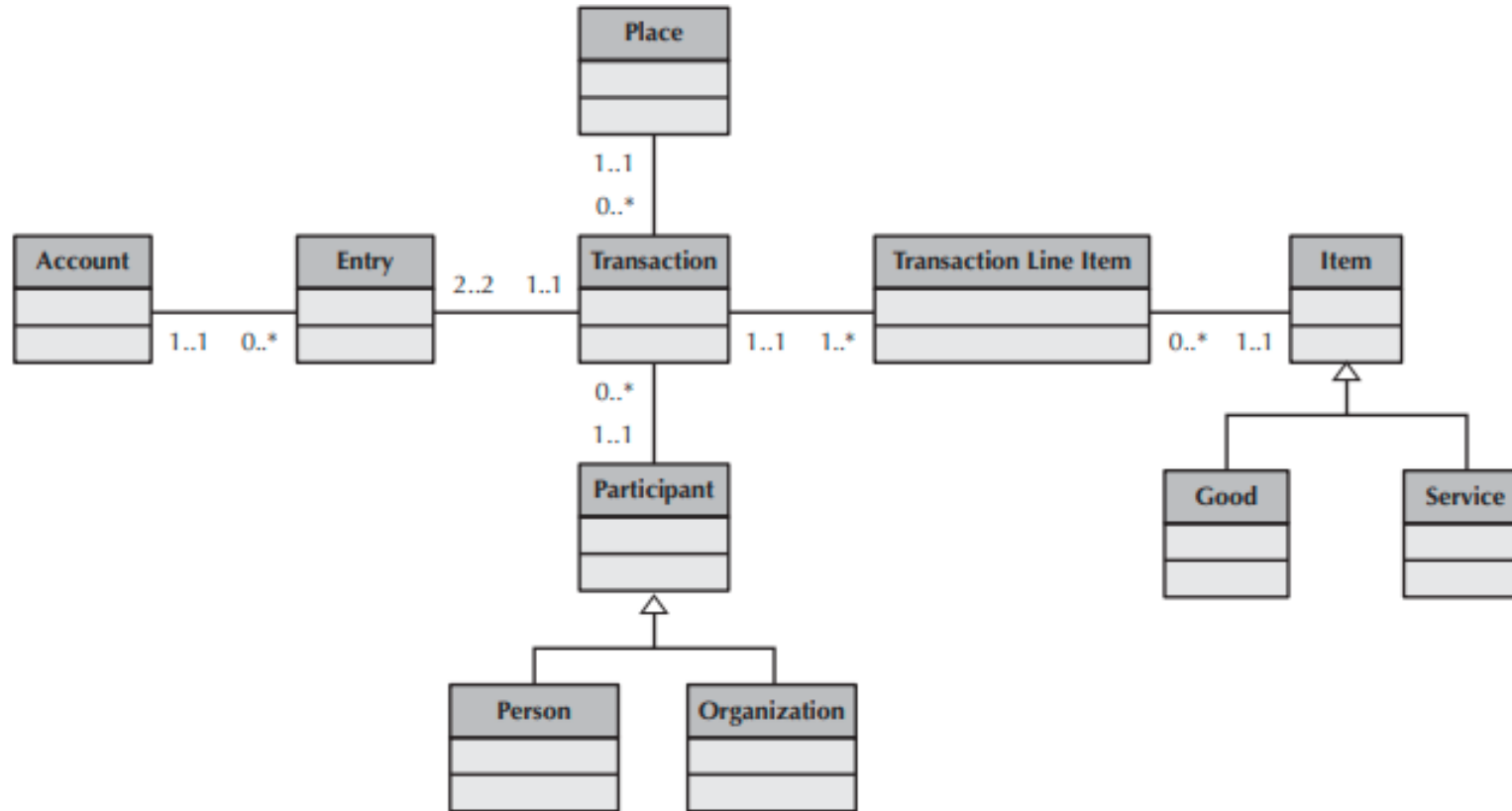


FIGURE 5-4 Sample Integration of Sample Patterns

- ❖ Structural models
- ❖ Object identification
- ❖ **Class diagrams**
- ❖ Structural models using class diagram

Sample Class Diagram

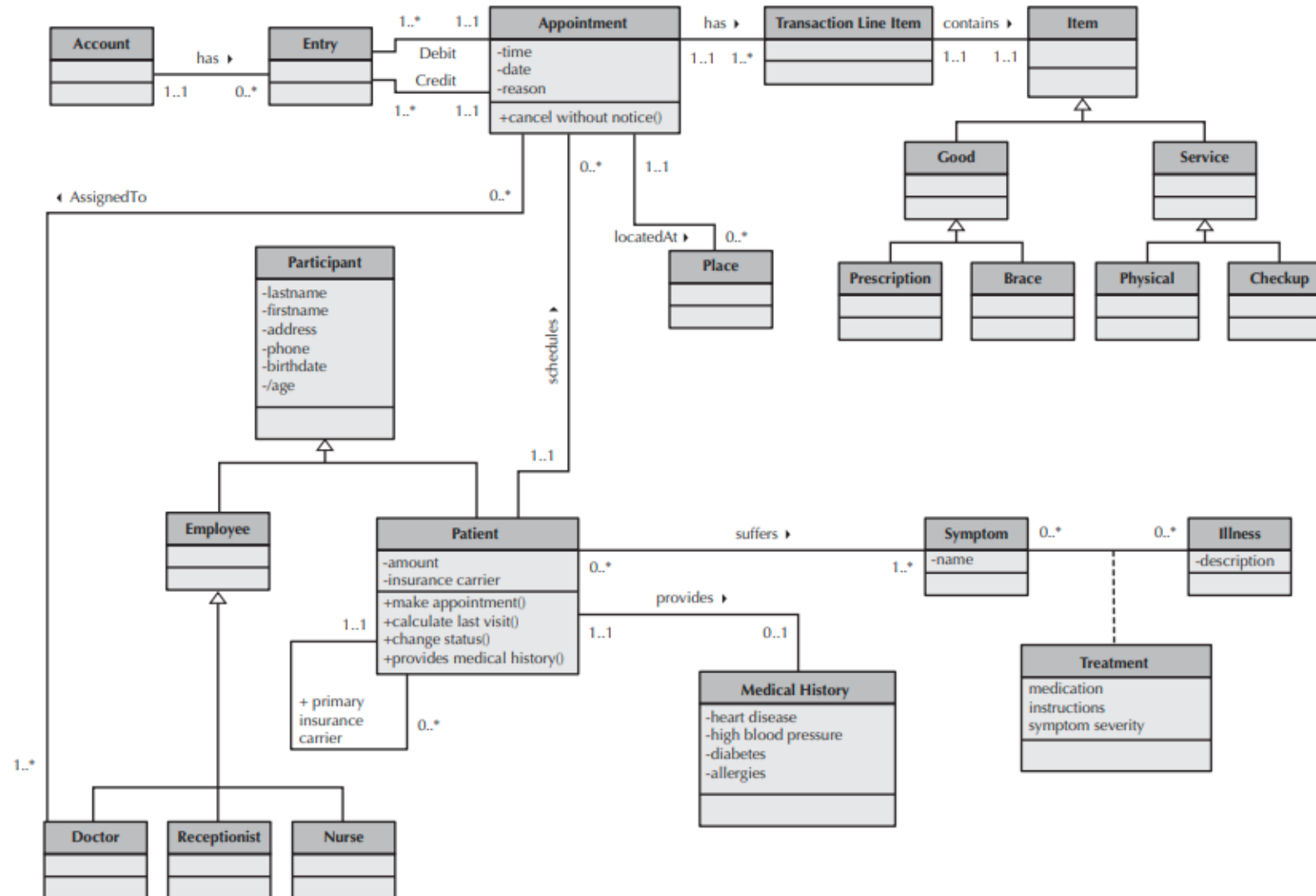
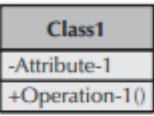
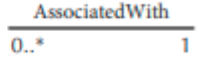
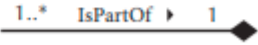


FIGURE 5-7 Sample Class Diagram

Class Diagram Syntax

- Class
 - Attribute
 - Operation
- Relationship
 - Association
 - Generalization
 - Aggregation
 - Composition
- Multiplicity

A class: • Represents a kind of person, place, or thing about which the system will need to capture and store information. • Has a name typed in bold and centered in its top compartment. • Has a list of attributes in its middle compartment. • Has a list of operations in its bottom compartment. • Does not explicitly show operations that are available to all classes.	 <pre> classDiagram class Class1 { -Attribute-1 +Operation-1() } </pre>
An attribute: • Represents properties that describe the state of an object. • Can be derived from other attributes, shown by placing a slash before the attribute's name.	attribute name /derived attribute name
An operation: • Represents the actions or functions that a class can perform. • Can be classified as a constructor, query, or update operation. • Includes parentheses that may contain parameters or information needed to perform the operation.	operation name ()
An association: • Represents a relationship between multiple classes or a class and itself. • Is labeled using a verb phrase or a role name, whichever better represents the relationship. • Can exist between one or more classes. • Contains multiplicity symbols, which represent the minimum and maximum times a class instance can be associated with the related class instance.	 <pre> classDiagram class1 "0..*" -- "1" class2 : AssociatedWith </pre>
A generalization: • Represents a kind-of relationship between multiple classes.	 <pre> classDiagram class1 -- > class2 </pre>
An aggregation: • Represents a logical a-part-of relationship between multiple classes or a class and itself. • Is a special form of an association.	 <pre> classDiagram class1 "0..*" -- "1" class2 : IsPartOf </pre>
A composition: • Represents a physical a-part-of relationship between multiple classes or a class and itself. • Is a special form of an association.	 <pre> classDiagram class1 "1..*" -- "1" class2 : IsPartOf </pre>



Multiplicity (Bội số quan hệ)

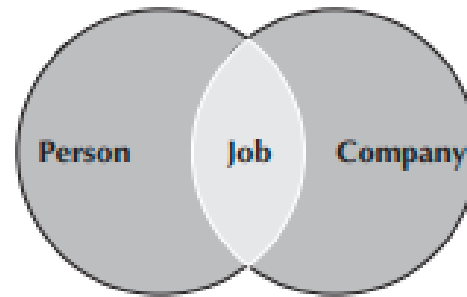
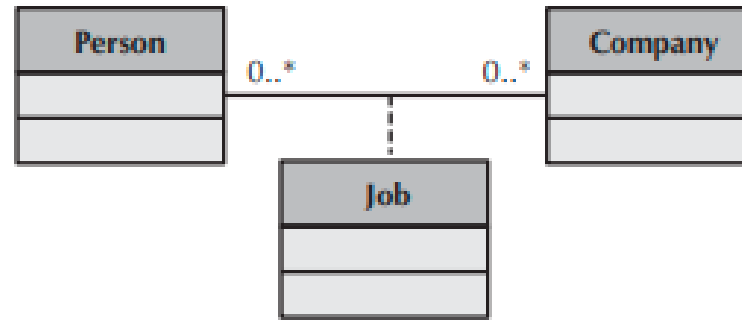
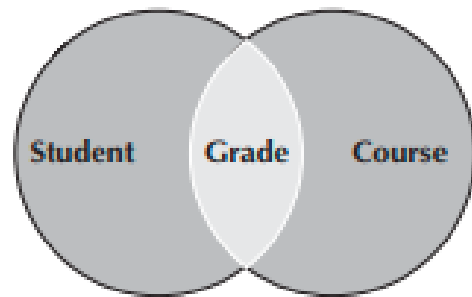
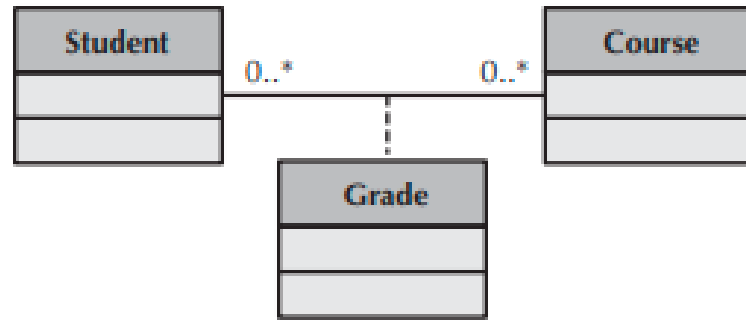


Exactly one	1	<pre> graph LR Department -- 1 --- Boss </pre>	A department has one and only one boss.
Zero or more	0..*	<pre> graph LR Employee -- 0..* --- Child </pre>	An employee has zero to many children.
One or more	1..*	<pre> graph LR Boss -- 1..* --- Employee </pre>	A boss is responsible for one or more employees.
Zero or one	0..1	<pre> graph LR Employee -- 0..1 --- Spouse </pre>	An employee can be married to zero or one spouse.
Specified range	2..4	<pre> graph LR Employee -- 2..4 --- Vacation </pre>	An employee can take from two to four vacations each year.
Multiple, disjoint ranges	1..3,5	<pre> graph LR Employee -- 1..3,5 --- Committee </pre>	An employee is a member of one to three or five committees.

Sample Association



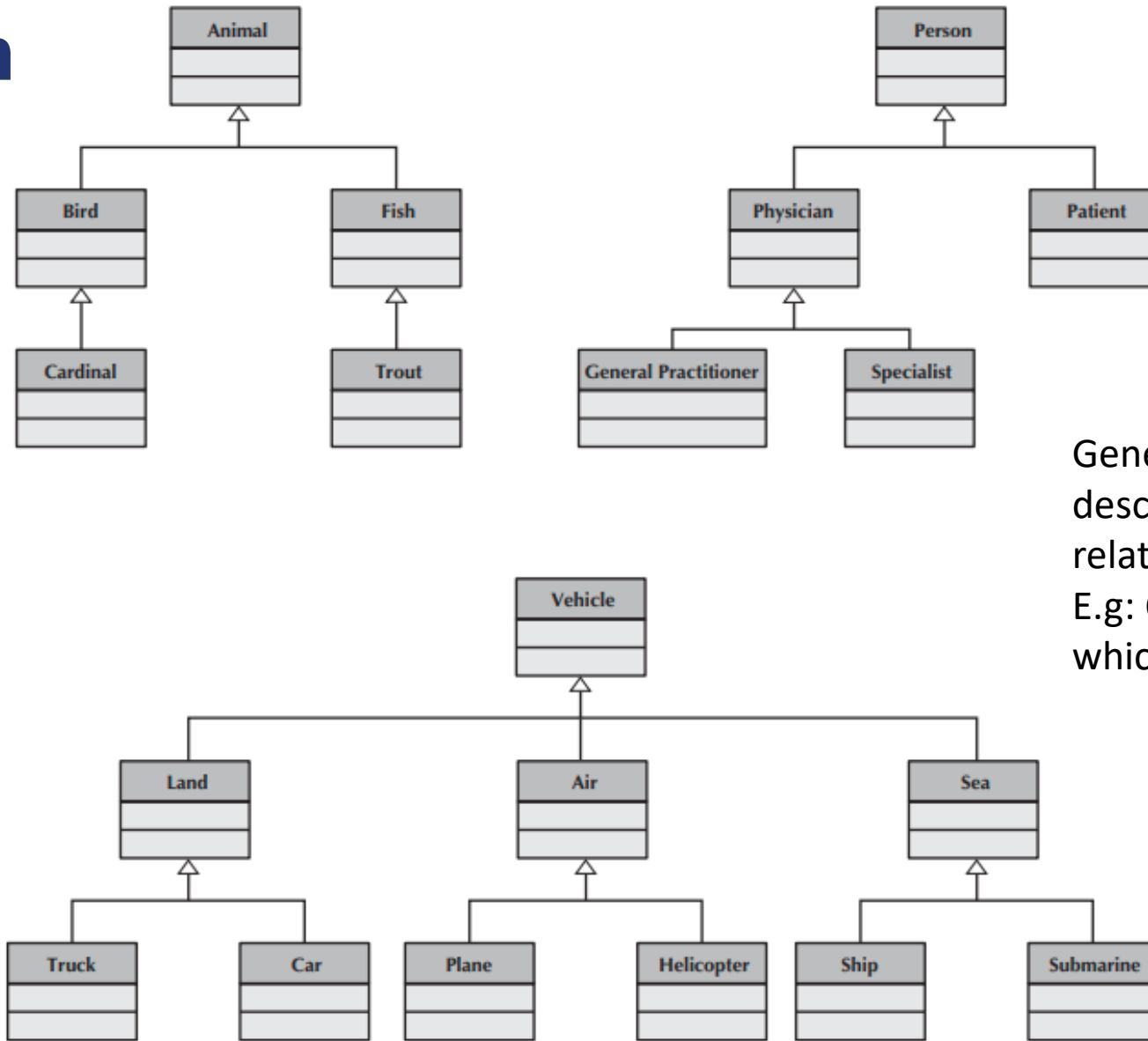
Multiplicity



Generalization



A generalization association shows that one class (subclass) inherits from another class (superclass)



Generalization relationship describe “is a kind of” relationship
E.g: Cardinal is a-kind-of Bird, which is a-kind-of Animal

FIGURE 5-12 Sample Generalizations

Aggregation

An aggregation association is used when classes actually comprise other classes.

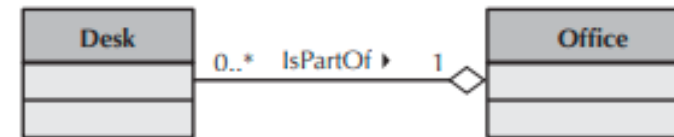
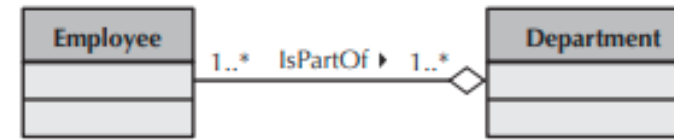


FIGURE 5-13
Sample Aggregation
Associations



FIGURE 5-14
Sample Composition
Associations

Simplifying Class Diagrams



- ❖ When a class diagram is fully populated with all the classes and relationships for a real-world system, the class diagram can become complex. It is necessary to simplify the diagram in some ways:
 - ❖ Show only concrete classes.
 - ❖ View subset of the class diagram
 - ❖ Group classes into packages

Object Diagrams

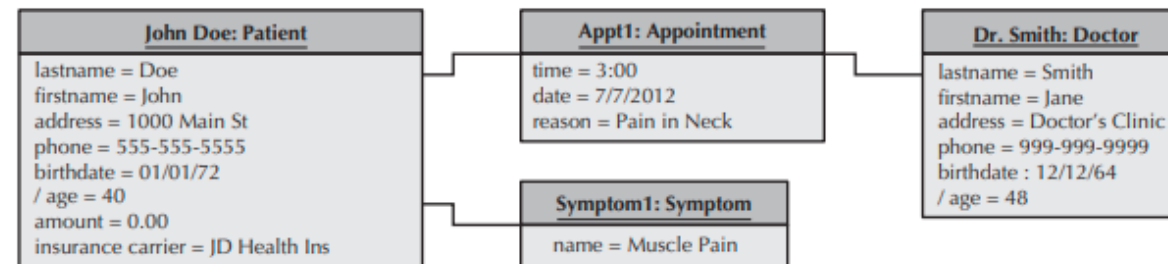
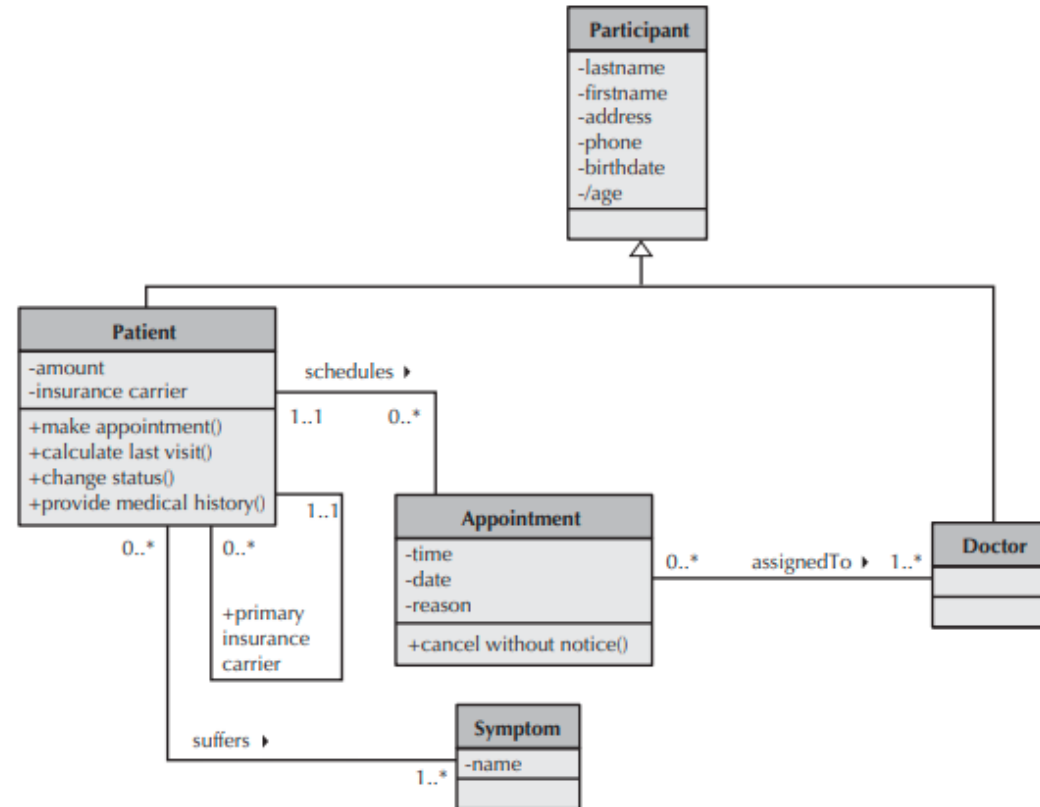


FIGURE 5-15 Sample Object Diagram

- ❖ Structural models
- ❖ Object identification
- ❖ Class diagrams
- ❖ Structural models using class diagram

Creating a structural model



- ❖ Creating a structural model is an incremental and iterative process whereby the analyst makes a rough cut of the model and then refines it over time
- ❖ Structural models can become quite complex. In fact, there are systems that have hundreds of classes
- ❖ Performing textual analysis on the use-case descriptions to find classes, its attributes and operation
 - ❖ The flow of events of the use-case description were written in a special form called Subject–Verb–Direct-Object–Preposition–Indirect object (SVDPI)
- ❖ Identify the relationship between classes

Class Diagrams for the Library System

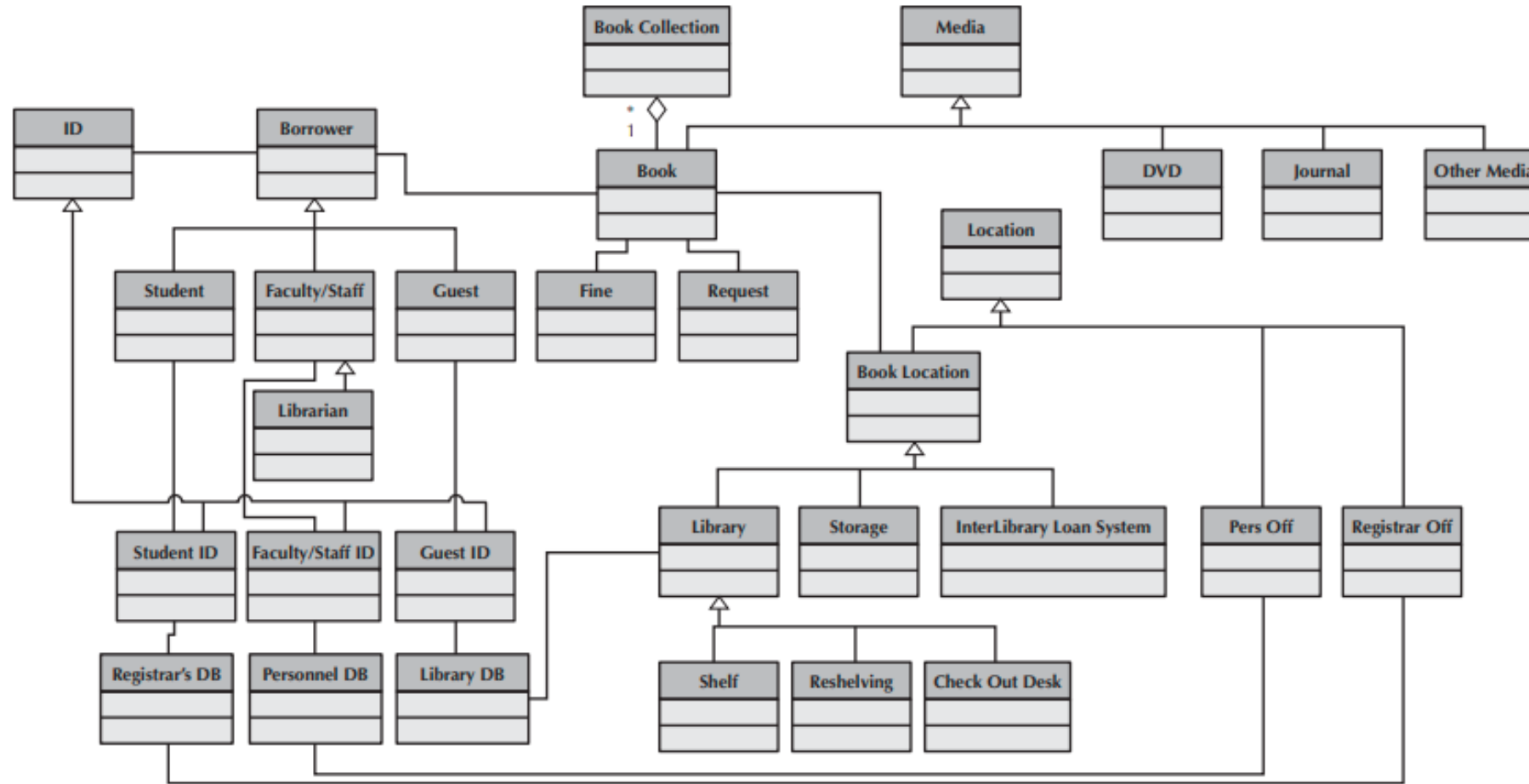


FIGURE 5-22 First-Cut Class Diagram for the Library Book Collection System

Simplifying Class Diagrams for the Library System

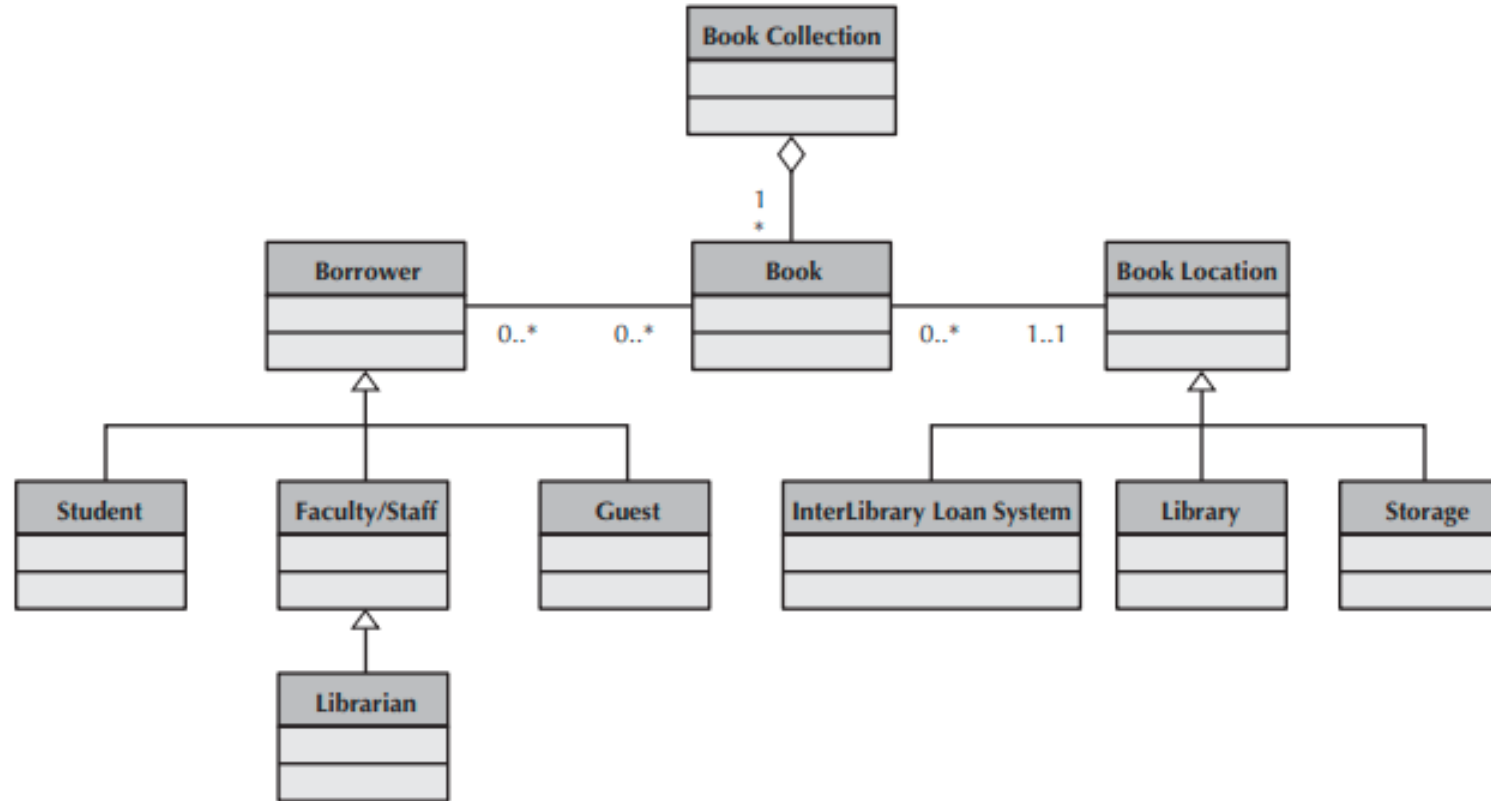


FIGURE 5-23 Second-Cut Class Diagram for the Library Book Collection System

Refine Class Diagrams for the Library System

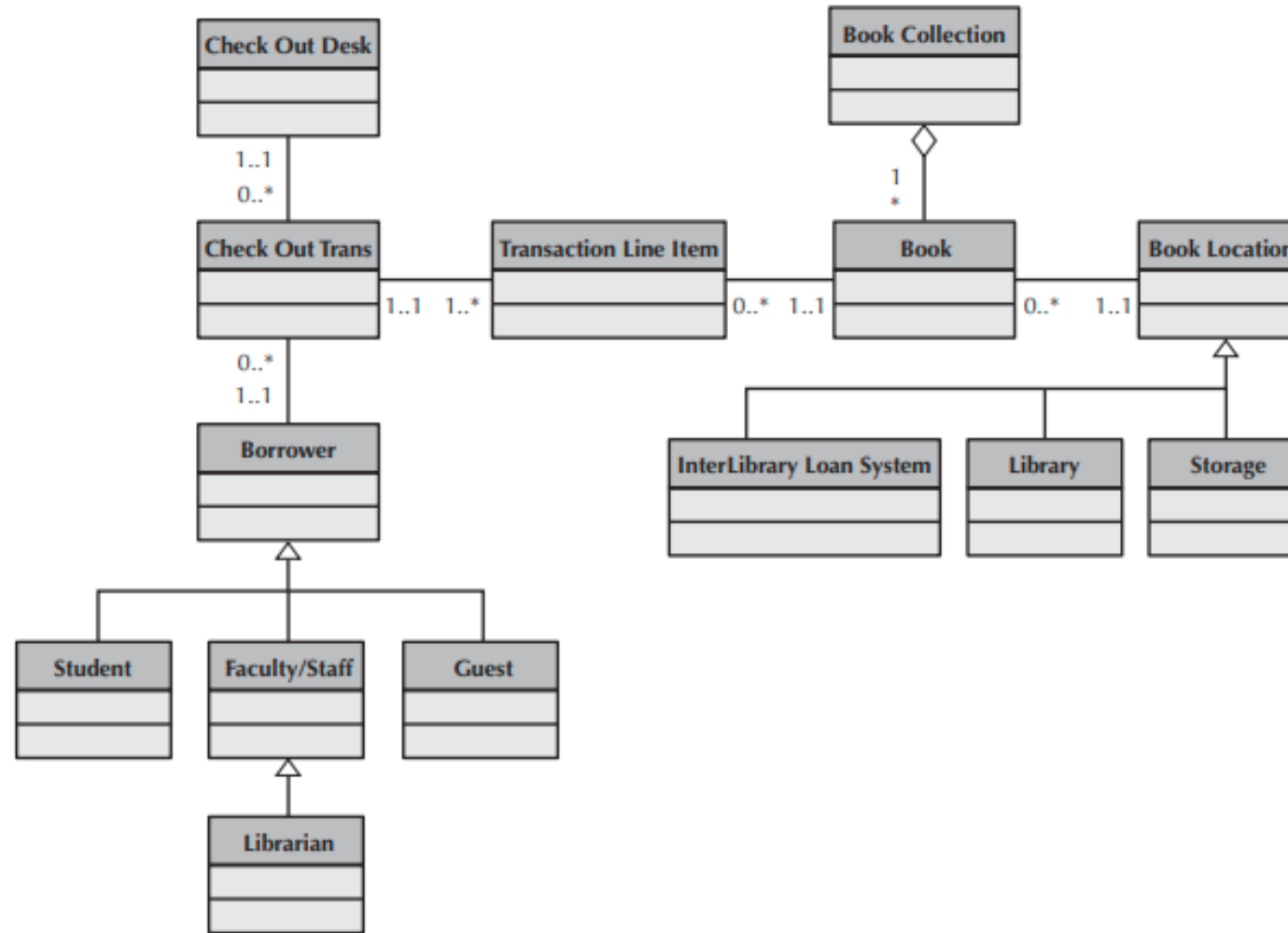


FIGURE 5-24 Class Diagram with Incorporated Pattern for the Library Book Collection System

Summary



- ❖ Describe the purpose of a structural model
- ❖ Understand class diagram syntax
- ❖ Create a structural model with class diagrams



PHENIKAA UNIVERSITY
Faculty of Computer Science

Q&A